# PDF Tool

A Sejda-style online PDF workspace

Edit, convert, organize, compress, redact, OCR and sign PDF files in the browser.

Live: pdftool-web.onrender.com · Source: github.com/mukeshroyhub/pdf_tool

## What it is

PDF Tool is a full-stack web application that lets users upload documents and run a wide range of PDF operations entirely online — no desktop software needed. Users get an account (or a one-tap guest session), a private file workspace with a storage quota, an activity timeline, and a set of editing and conversion tools comparable to iLovePDF, Sejda and PDF24. Everything runs same-origin in the browser through a Next.js frontend that proxies to a TypeScript API.

## Feature set

| | |
|---|---|
| **Accounts** | Email + password sign-up, Google sign-in, and one-tap guest mode. Rotating secure sessions; per-user storage quota (1 GB standard, 100 MB guest). |
| **Upload & manage** | PDF, image and Office files up to 100 MB each. File list with search, favorites, rename, delete, and a storage meter. |
| **View** | In-browser PDF viewer powered by pdf.js. |
| **Edit / annotate** | Add text, highlights, whiteout, rectangles, ellipses, lines and freehand ink. Add watermarks. |
| **Organize pages** | Reorder, rotate, delete, duplicate and insert blank pages; replace pages from another PDF. |
| **Merge & split** | Combine multiple PDFs into one; split a PDF into parts by page ranges. |
| **Compress** | Reduce file size at low / medium / high / custom quality levels. |
| **Convert** | PDF → PNG/JPG, images → PDF, Office (Word/Excel/PPT) → PDF, PDF → DOCX/PPTX, and PDF → Excel (table extraction). |
| **OCR** | Turn scanned PDFs into searchable text (English, German, French, Spanish, Hindi). |
| **Redact** | True raster redaction that destroys the underlying content, plus text / watermark removal. |
| **Forms** | Fill in PDF form fields. |
| **Batch & history** | Run operations across multiple files; activity timeline logs every action. |

## Technology stack

- **Frontend** — Next.js 15, React 19, Tailwind CSS, Radix / shadcn-style UI, TanStack Query, react-hook-form, pdf.js.
- **Backend** — Node.js + Express + TypeScript, with Zod schemas shared between client and server for one source of validation.
- **PDF engine** — pdf-lib, pdf.js, @napi-rs/canvas (page rendering), sharp (images), ExcelJS (PDF→Excel), LibreOffice (Office conversion), Tesseract (OCR).
- **Data** — Prisma ORM on Neon PostgreSQL (production) / SQLite (dev).
- **Storage** — Supabase object storage (S3-compatible) in production; local disk in dev, behind one swappable driver.

- **Auth** — JWT access tokens + rotating refresh tokens (httpOnly cookie, hashed, reuse-detection) and Google OAuth.
- **Delivery** — Docker images on Render (web + API), with GitHub Actions CI running typecheck, tests and build on every push.

## How to use it

- Open the site and sign up, sign in with Google, or click **Continue as guest**.
- Drag a file onto the upload area (or click to browse) — PDF, image or Office document.
- Open the file and pick a tool: edit, organize, merge, split, compress, convert, OCR or redact.
- Download the result, or keep it in your workspace. The activity panel tracks everything you do.

## How it fits together

The browser talks to a single origin: the Next.js web app serves the UI and transparently proxies all `/api/*` calls to the Express API, so authentication cookies stay first-party and there is no cross-origin setup. The API validates every request with Zod, stores metadata in PostgreSQL, and streams file bytes to and from object storage — which means uploads survive server restarts and the app can scale beyond a single machine.

---

Generated 14 Jul 2026